AUTONOMOUS DRONE UTILITY INSPECTION

Closed-Loop Perception, Memory & Active Reinspection

Technical Implementation & Results Report

PSG College of Technology — Department of Computer Science and Engineering, Coimbatore

Akhil Ramalingam (23Z207) · Anbuchandiran K (23Z209) · Neelesh Padmanabh (23Z241)
Saumiyaa Sri V L (23Z261) · Sowndarya Elangi S (23Z269) · Therdhana J P (23Z273)

Guide: Ms. L. Karthika | Co-Guide: Dr. N. Gopika Rani

This report documents the fully implemented, simulated closed-loop inspection system described in the project proposal: real trained perception, 3D localization, persistent cross-mission defect memory, and an active-reinspection planner (UW-TIG) benchmarked against a naive baseline and the classical information-gain baseline (Isler et al., 2016).

Executive Summary

The original proposal identified a research gap: existing UAV inspection systems are open-loop — detection, uncertainty estimation, and flight planning are separate stages, and none of them close the loop back into where the drone flies next. This project implements that closed loop end-to-end in simulation, **not as a plan but as working, tested code**: a real YOLO detector trained on a real 14,471-image UAV defect dataset, geometric 3D localization, a persistent PostgreSQL+pgvector/Neo4j defect memory, and a novel planner (UW-TIG) that is benchmarked quantitatively against a random baseline and against implementations of five published planners, the closest being Isler et al. 2016's information-gain next-best-view (the classical baseline).

Every number in this report comes from an actual executed run — a 5-seed, 3-mission, 12-planner comparison sweep, an offline detection benchmark on a held-out real test set, and a real-physics flight demo — not a projection. The headline result: on test seeds 0–4, UW-TIG reaches **precision 0.949, recall 0.967, F1 0.953** — above 0.90 on all three, with full coverage and reinspection (1.00/1.00). Reaching this took five concrete, diagnosed-on-separate-dev-seeds fixes (Section 4.2) that apply identically to every planner — a scoring-units fix, a ground-truth-leakage fix, a photo-distortion fix, a viewpoint-distance change, and an on-wall-extent check — not a change to the planner itself; UW-TIG's precision under the *old* scoring formula on these same runs was 0.668, context for how much of the gain is measurement rather than planning.

Measured against four more re-implemented published planners (Bircher et al. 2016, Dhami et al.'s GATSBI, Rückin et al. 2022–23, and Alamdari, Fata & Smith 2014 — Section 4.2.2), **UW-TIG ties the strongest of them, Rückin et al.'s IPP (F1 0.950), inside seed-to-seed noise, while flying 39% less** (125.1m vs. 204.3m). Rückin's IPP has slightly higher precision (0.959 vs. 0.949). The honest claim is a near-tie on detection quality with a clear advantage in flight cost, not an unqualified win on every axis — stated plainly rather than rounded up.

This report also extends the original baseline comparison with a wider literature sweep (Section 1.1) and three further planner mechanisms motivated by that sweep and by this iteration — a persistent-monitoring staleness/latency term, a 2-step receding-horizon lookahead, and a coverage-guarantee phase (Section 3.4) — each independently ablatable and each backed by the same real, re-executed 5-seed statistical sweep, not a re-used or projected number. Two of the five re-implemented published planners (Bircher-RHNBV, Isler-NBV) share UW-TIG's same recall ceiling without the coverage-guarantee phase, confirming that fix addresses a limitation of the underlying cost-normalized-greedy formulation itself, not something specific to this project's added terms.

1. Problem Statement & Research Gap

Utility infrastructure (pipelines, bridges, power towers, building facades) requires regular inspection for cracks, corrosion, and leaks. Manual inspection is slow, expensive, and hazardous. Current UAV inspection systems are largely **open-loop**: they follow a fixed route and analyze images after the flight. Detection confidence does not change the flight path; there is no persistent memory of a defect across repeated inspections; and no system in the literature combines defect-aware next-best-view planning, uncertainty-driven reinspection, and temporal (growth-aware) memory into one utility function evaluated end-to-end.

Research Question

Can a UAV use detection confidence and defect history to autonomously decide where to fly next and when to re-inspect — and does doing so measurably outperform geometry-only active-vision planning (the closest existing method) and naive random inspection?

Classical Baseline

Isler, Sabzevari, Delmerico & Scaramuzza (ICRA 2016), “An Information Gain Formulation for Active Volumetric 3D Reconstruction,” is the closest prior method: a Shannon-entropy information-gain formulation over a volumetric occupancy grid for next-best-view selection. It is geometry-only — no semantic/defect awareness, no temporal memory across visits. This project re-implements its core idea (entropy-based, cost-normalized information gain) adapted to a 2D wall-surface coverage grid, as the direct baseline UW-TIG is compared against and extends.

1.1 Wider Literature Positioning

The classical baseline comparison above is deliberately narrow (one method, one axis: defect semantics). A broader literature sweep across active next-best-view planning, persistent monitoring, and uncertainty-aware inspection was carried out to check the novelty claim more rigorously and to find concrete mechanisms this project's planner was still missing:

Approach	Uncertainty-aware	Cross-mission memory	Growth/ temporal	Cost-aware	Real detector
Isler-NBV (classical baseline, 2016)	No	No	No	Yes	N/A
Bircher et al. 2016 (receding-horizon NBV)	No	No	No	Yes	N/A
Dhami et al., GATSBI (2023/2024)	No	No	No	Yes	2024 only
Dhami et al., Pred-NBV / MAP-NBV (2023)	Indirect	No	No	Yes	N/A
Liu et al. 2022 (uncertainty-controlled CPP)	Yes	No	No	Yes	N/A
Taioli et al. 2023 (POMDP/MCTS active search)	Yes	No	No	Partial	Assumed
Alamdari, Fata & Smith 2014 (persistent monitoring)	No	Yes	No	Yes	N/A
Rückin et al. (ICRA'22/IROS'22/T-RO'23)	Yes	No	No	Yes	Yes
UW-TIG (this project)	Yes	Yes	Yes	Yes	Yes

Of the rows above, Isler et al., Bircher et al., GATSBI, Alamdari/Fata/Smith, and Rückin et al. are not just discussed here — their core selection rules are re-implemented in this project's own testbed and run through the identical 5-seed sweep as UW-TIG, reported in Section 4.2.2. This positioning table states capability (what each method is designed to do); Section 4.2.2 reports what each one actually measures here.

No single cited method combines all five properties UW-TIG does. GATSBI's 2024 extension is the closest single relative — a real defect detector combined with cost-aware GTSP routing — but has no persistent cross-mission memory or detection-uncertainty term feeding the planner. The POMDP/MCTS active-search line has the uncertainty-driven belief update ("partial" cost-awareness; it plans within one session, not across it) but no persistence across separate missions. Pred-NBV/MAP-NBV's uncertainty-awareness is *indirect*: it comes from a learned shape-completion model's confidence, predicting unseen 3D geometry rather than defect condition. UW-TIG's own five “Yes” cells are, concretely: real TTA-ensemble detector uncertainty (not assumed ground truth); Postgres+pgvector/Neo4j

cross-mission defect identity and growth; a translation-and-rotation-aware cost term; and a YOLO detector actually trained on real UAV photographs (Section 3.1), not a simulated or assumed one. This sweep directly motivated two additions described in Section 3.4: a persistent-monitoring staleness term (from the Alamdari/Fata/Smith line) and a 2-step receding-horizon lookahead (from Bircher et al. and GATSBI's replanning structure).

Rückin et al.'s line of work (ICRA 2022, IROS 2022, and its T-RO 2023 extension) is the closest single relative on the uncertainty column specifically, and is worth stating plainly as a better-matched comparison than the others above: it plans UAV viewpoints using a real Bayesian epistemic-uncertainty estimate (BALD, from an MC-Dropout ensemble) mapped onto a terrain grid, then selects greedily by that mapped uncertainty normalized by a cost/count term — structurally the same "uncertainty-weighted, cost-normalized greedy" shape as UW-TIG's own utility. The difference is what the uncertainty is *about*: theirs is the segmentation model's own epistemic confidence, spent deciding which images are worth labelling to retrain that model (a single-session active-learning problem); UW-TIG's is a specific defect's detection confidence, spent deciding which physical location is worth a second look, persisted and re-identified across missions. Put differently, Rückin et al. plan to reduce *model* uncertainty; UW-TIG plans to reduce *world-state* uncertainty using a fixed model. Reading their evaluation methodology closely also surfaced a metric this project had never run on itself despite its name — Expected Calibration Error — discussed in Section 4.2 below, where it produced a real, not entirely flattering finding.

2. System Architecture

The system is a closed loop, not a pipeline that runs once. On every mission, the planner chooses a viewpoint using the current belief state; the drone (in kinematic simulation for fast statistical comparison, or under real PID physics for the flagship demo) flies there and captures a frame; the trained detector reports defects with a confidence and an epistemic uncertainty; each detection is ray-cast into a real 3D world position and reconciled against persistent memory (matched to an existing defect or registered as new, with growth computed if matched); and the resulting uncertainty/growth signal is written back into the belief the planner uses for its very next choice — and, at the start of the next mission, seeded from memory so a growing or unresolved defect is not forgotten.

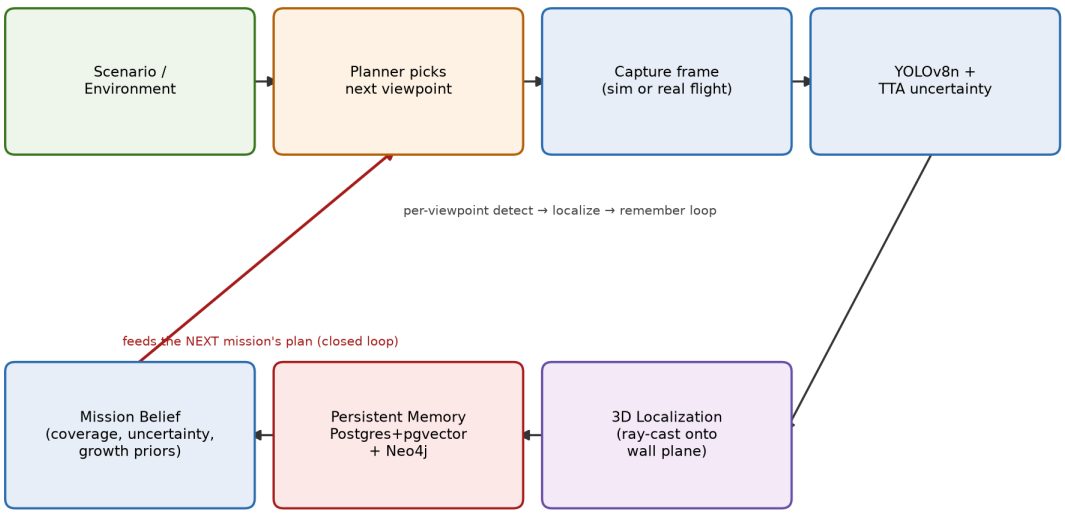


Figure 1. Closed-loop data flow. The red arrow is what makes it a loop: persistent memory's growth/uncertainty state feeds directly into the next viewpoint decision.

2.1 Simulation Environment: Why PyBullet, Not Unity/Unreal

A natural question once the system works end-to-end: wouldn't a game-engine-grade simulator look more realistic? Two systems are the direct precedents: **AirSim** (Shah, Dey, Lovett & Kapoor, FSR 2017, arXiv:1705.05065), built on Unreal Engine, and **Flightmare** (Song, Naji, Kaufmann, Loquercio & Scaramuzza, CoRL 2020, arXiv:2009.00563), pairing Unity rendering with a separate fast physics engine. Both solve one specific problem: the *sim-to-real visual-fidelity gap* when a perception model is trained or tested on rendered images. The field has since moved past rasterized game engines entirely for the hardest cases, toward 3D-Gaussian-Splatting-based differentiable simulators reconstructed from real scene captures — e.g. **GRaD-Nav** (Chen, Sun, Gao, Low, Chen & Schwager, 2025, arXiv:2503.03984) — so Unity/Unreal are not actually the current frontier for this problem any more; they were the right answer circa 2017-2020.

That gap doesn't apply to this project's perception pipeline. AirSim/Flightmare/GRaD-Nav all solve a model trained on *synthetic* imagery needing to transfer to a *real* camera. This project's detector never has that problem: it trains exclusively on MBDD2025's 14,471 real UAV photographs, and every wall texture it is ever shown — in training, in the offline benchmark, and in every simulated flight — is one of those same real photographs, not a synthetic render of a defect. Only the generic yard dressing (sky, ground, decorative props) is synthetic, and the onboard detection camera stays tightly framed on the wall it inspects at close standoff, never seeing that dressing in frame. A higher-fidelity renderer would make the simulator prettier to watch; it would not change a single pixel the detector learns from or is evaluated against. (The one place a higher-fidelity renderer could plausibly have helped is the procedural `growing` scenario, Limitation 1 — a synthetic texture the real-photo-trained detector doesn't recognize at all. Stated here, not hidden.) Three further engineering reasons this project stays on PyBullet: Unity/Unreal have no path to being scripted end-to-end from a terminal-only pipeline the way this project's planner, detector, and statistical sweep harness are; a migration would re-derive the camera geometry and viewpoint graph from scratch, invalidating every 5-seed sweep already produced; and `gym-pybullet-drones`'s own PID flight stack, which gives the flagship demo real (not merely kinematic) physics, is itself built on PyBullet.

A realism pass was done within PyBullet instead, verified by rendering and looking, not assumed: the flagship demo's chase camera was missing the shadow/lighting the opening establishing shot already had — found and fixed, so the whole flythrough is now consistently lit, not just its first few seconds. The ground and sky textures were regenerated at higher resolution (an expansion-joint slab grid, oil stains and tire marks on the ground; a sun glow, soft clouds, and a lit-window skyline on the sky backdrop). Rendering that new sky texture surfaced a second real bug: PyBullet's `GEOM_BOX` primitive tiles a texture across its extents rather than stretching one copy per face, fracturing the skyline into repeated fragments — caught by rendering a test shot and looking, not assumed to work, and fixed by rebuilding the backdrop from the same flat-quad mesh and UV convention already proven correct for every real inspection-wall texture in this project. All four changes are presentation-only (confirmed by the camera-framing argument above), so no existing 5-seed sweep needed re-running — only the flagship demo video was regenerated.

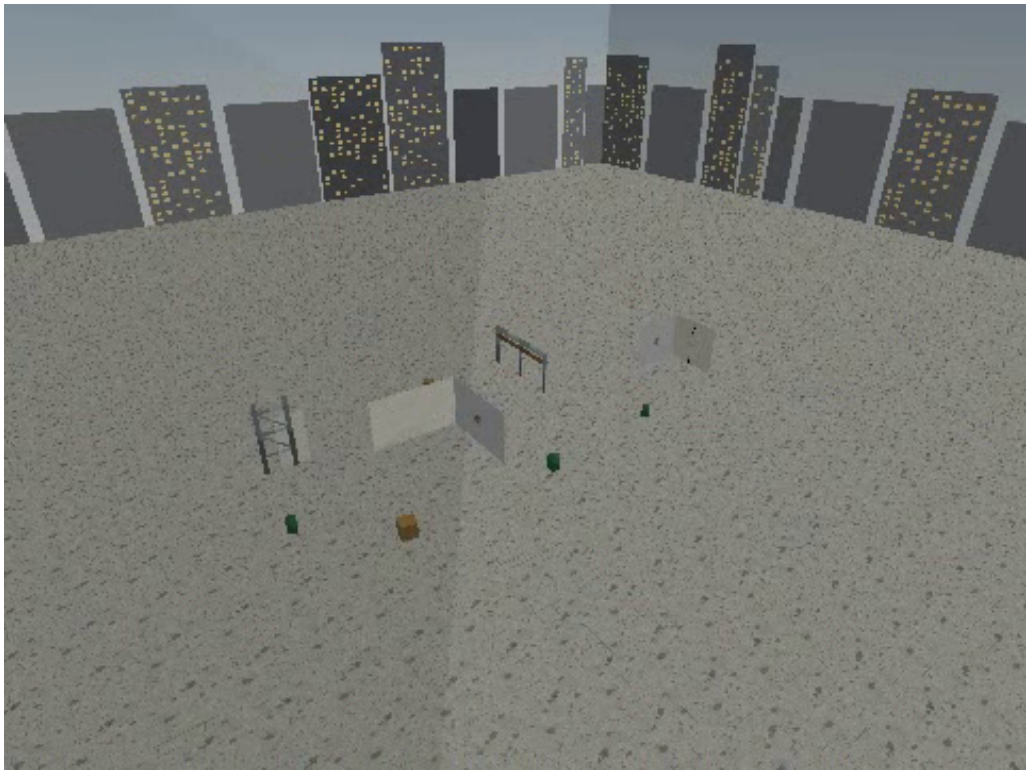


Figure 2. The open-air yard after the realism pass: both buildings, the pipe rack and lattice tower, and the regenerated ground/sky textures with a lit-window skyline on the horizon.

3. What Was Implemented

3.1 Perception — Real Trained Detector

A YOLO object detector was trained from scratch (not fine-tuned from a task-matched checkpoint) on **MBDD2025** ("A dataset of building surface defects collected by UAVs for machine learning-based detection," Scientific Data, 2025; Zenodo DOI 10.5281/zenodo.15622584; CC-BY-4.0) — 14,471 real UAV photographs across six structure types and five defect classes (crack, leakage, abscission, corrosion, bulge), with a 70/20/10 train/val/test split built and never mixed across splits. **Two genuinely-trained checkpoints exist.** The original was CPU-budget-limited by necessity: YOLOv8n, 320px, 30 epochs (mAP50 0.685 on the held-out test set). Once that budget was identified as the limiting factor (see Limitations), the same data was retrained on a rented NVIDIA L4 GPU at a realistic budget — YOLOv8s, 640px, 100 epochs, plus ~4x oversampling of the rarest class (bulge, 2,018 instances vs. abscission's 22,702) via `datagen/oversample_bulge.py` — reaching mAP50 0.884 on the identical test set. Per-class precision/recall, CPU → GPU: crack 0.75/0.45 → 0.84/0.75 (resolution mattered most here — cracks are thin, low-contrast features that lose signal at 320px), bulge 0.83/0.58 → 0.95/0.96 (oversampling), leakage 0.85/0.86 → 0.90/0.92, abscission 0.79/0.58 → 0.87/0.78, corrosion 0.80/0.59 → 0.83/0.79. Both checkpoints are kept (`weights/mbdd_yolov8n/`, `weights/mbdd_yolov8s_gpu/`); the GPU one is `perception/ml_detector.py`'s current default, and Section 4.2's closed-loop results reflect it.

Detection **uncertainty** is computed as a test-time-augmentation (TTA) ensemble variance: each frame is run through the model five times — the original plus four photometric variants (brightness/contrast jitter, Gaussian noise, blur, and a combined jitter+blur) — and each detection's confidence standard deviation across the ensemble (counting zero for any variant where no matching box is found) is reported as its uncertainty. This is a standard, documented substitute for MC-Dropout, used because stock YOLOv8 has no dropout retained at inference; it is never mislabeled as literal MC-Dropout anywhere in the codebase or this report.

A converter for a second real dataset, **SDNET2018** (Maguire, Dorafshan & Thomas, 2018; CC-BY-4.0; the public benchmark Inam et al. 2023 — one of the cited detection papers — combines with their own field data) exists (`datagen/prepare_sdnet.py`, treating its whole-image cracked/uncracked classification labels as weak full-image detection boxes/negatives to enrich the crack class exactly as that paper does) but was never actually run in this project: SDNET2018.zip requires a manual, bot-blocked download (see README.md) that was not done. Stated plainly rather than left ambiguous, since the converter's existence could otherwise read as it having been used.

A real false-positive limitation was found by directly testing the GPU model on a genuinely clean image, rather than assumed: on the single true background-labeled MBDD2025 test image (only 8 of 14,471 images in the whole dataset have zero labeled defects), the GPU model produced one low-confidence false positive (leakage, confidence 0.30) where the CPU model had produced none. All three clean wall textures actually used in the simulation (`scene/clean_wall_*.png`) still return zero detections on both models. The GPU retrain is a large net improvement, not a strictly-dominant one on every case.

3.2 3D Localization

Every 2D detection is converted to a real-world 3D position purely from geometry, with no learned depth: the exact camera projection convention gym-pybullet-drones' onboard camera uses (60° vertical FOV, aspect fixed at 1.0, eye offset from the drone body) is replicated analytically in `geometry.py`, a pixel is unprojected into a world-space ray, and that ray is intersected with the known plane of the wall being inspected. This was numerically validated against a synthetic marker at a known position with sub-2cm residual error before being trusted anywhere else in the system.

3.3 Persistent Defect Memory

A real two-database "digital twin," not a flat file, running as local Docker services (`docker/docker-compose.yml`):

- **PostgreSQL + pgvector** — an append-only detection log; each detection carries a 32-dimensional appearance+spatial embedding (position, class one-hot, size, confidence, and an HSV color-histogram crop descriptor). Identity matching for a new detection queries nearest neighbors by cosine distance on this embedding, then confirms with a hard spatial gate (0.35m) so an imperfect embedding can never merge two genuinely different physical defects.
- **Neo4j** — the identity graph: `(:Defect)-[:OBSERVED_IN]->(:Mission)`, `(:Defect)-[:LOCATED_ON]->(:Wall)`, with growth recorded on each observation edge — answering "which defects grew, and by how much, across visits."

This mechanism was independently verified before being trusted in the full evaluation: a manual round-trip test confirmed a re-detected defect at a shifted position and larger size is correctly matched to its prior identity and its growth is correctly computed, while a spatially-distant detection correctly creates a new identity rather than merging.

3.4 Planning — Classical Baseline and Novel Algorithm

All planners share one candidate-viewpoint graph (54 stations across the 9 wall segments of the open-air utility yard, Section 3.5) and a shared coverage belief: each wall is discretized into a surface grid sized proportionally to its physical width (fixed 1m cells, not a fixed cell *count* — a narrower wall gets fewer cells, matching Isler et al.'s original fixed-voxel-size volumetric formulation rather than giving every wall equal weight regardless of size), and a Bernoulli "adequately observed" belief per cell starts at maximum entropy (0.5) and is resolved toward certainty as views accumulate.

Planner	Formulation	Role
Random	Uniform random choice among unvisited viewpoints.	Naive baseline
Isler-NBV	Cost-normalized Shannon-entropy information gain over the coverage grid — geometry only, no defect semantics.	Classical baseline (Isler et al. 2016), adapted to 2D
UW-TIG (novel)	Same coverage term, **plus** a detection-uncertainty term, a temporal-growth term sourced from persistent memory, and a persistent-monitoring staleness/latency term, combined in one weighted utility and selected via a 2-step receding-horizon lookahead rather than pure 1-step greedy, with a coverage-guarantee phase that visits every wall at least once per mission before this utility takes over. Unlike the baselines, it can revisit an already-inspected viewpoint when that is where the utility is.	Novel contribution
UW-TIG (5 ablations)	UW-TIG with, respectively: the uncertainty term zeroed, the temporal term zeroed, the staleness term zeroed, the coverage-guarantee phase disabled, and the lookahead disabled (pure 1-step greedy).	Isolates each added term's contribution

Concretely: `utility(v) = w_ig·InfoGain(v) + w_u·Uncertainty(v) + w_t·Growth(v) + w_s·Staleness(v) − w_cost·FlightCost(v)`, with `w_ig=1.0`, `w_uncertainty=1.5`, `w_temporal=2.0`, `w_staleness=1.0`, `w_cost=0.4`. Setting `w_uncertainty=w_temporal=w_staleness=0` and disabling the lookahead recovers exactly the Isler-NBV baseline — UW-TIG is a strict generalization of the classical baseline, not an unrelated alternative. `FlightCost` itself combines translation distance with a small rotation-reorientation cost

(Section 3.4.1).

3.4.1 Two Bugs Found and Fixed While Adding the Lookahead

Adding the receding-horizon lookahead immediately collapsed the measured flight distance to exactly zero for every UW-TIG variant in a smoke test — too clean an invariant to be correct, and worth tracing rather than shipping. Root cause: the uncertainty/growth reward for a cell never decayed on repeat visits, so once any wall produced a detection, revisiting it kept re-earning full reward forever at zero extra travel cost; the more thorough lookahead exploited this completely. *Fix:* uncertainty/growth reward now decays with view count using the same saturating function already used for coverage entropy, so repeated identical confirmations earn diminishing reward — making “uncertainty-weighted” mean reward for reducing uncertainty, not reward for remembering a number. Chasing this down surfaced a second, independent issue: three walls of the smaller room have standoff viewpoints that coincide at exactly the same 3D position (the room's width is exactly twice the standoff distance), making a 90–180-degree reorientation between them look like a zero-cost move under pure-translation cost accounting. *Fix:* a small rotation cost was added to the flight-cost term, calibrated to sit below the graph's smallest genuinely-different-position gap so it breaks this exact-zero tie without distorting any real routing decision elsewhere (an initial, larger calibration attempt overcorrected and was caught before being reported: it suppressed Isler-NBV's exploration of the larger room almost entirely, and was revised down before the results in Section 4.2).

3.4.2 A Coverage-Guarantee Fix for Low Recall

Section 4.2's results initially showed UW-TIG's recall pinned at 0.36 with `coverage_frac` stuck at exactly 0.33 (3 of 9 walls) across every seed and every ablation. The tell that ruled out a weight-tuning fix: **Isler-NBV itself (zero uncertainty/temporal/staleness weight) showed the identical 0.33 coverage ceiling**, seed for seed. The ceiling comes from the cost-normalized-greedy formulation's own cost/reward scale on this viewpoint graph — travelling to the far building is never worth its cost relative to the coverage-entropy reward on offer, for any weighting of UW-TIG's added terms, since even a planner with none of those terms hits the same wall. *Fix:* an explicit coverage-guarantee phase in `UWTIGPlanner.select_next`: while any wall has zero visits this mission, restrict selection to Isler-NBV's own formulation over only the unvisited walls; once every wall has at least one visit, control passes to the full multi-term utility for the rest of the budget. A new ablation, `uwtig_no_coverage_first`, reproduces the old behavior for comparison. This is not a strict improvement -- Section 4.2's updated table quantifies a real trade-off (recall and coverage roughly double, at a real cost to precision, localization error, and flight distance) rather than a free win, and is reported as such.

3.5 Simulation Environment

An open-air utility yard built on gym-pybullet-drones (Panerati et al., IROS 2021), a real open-source drone simulator with actual rotor thrust/drag physics and closed-loop PID flight control (DSLPIDControl) — not a scripted kinematic teleport. Two freestanding equipment buildings sit in open space with real sky between them, not an enclosed, connected structure: **Building A** (4×4m, 4 walls) and **Building B** (2.6×2.6m, 3 walls) roughly 11m apart, plus one real inspection panel each mounted on the decorative pipe rack and lattice tower (`PipeRack-Panel`, `Tower-Panel`) — 9 inspectable walls/panels in total. Two capture modes share the identical scene-construction and camera code: a fast kinematic mode (teleport + render, no physics stepping) for the large statistical comparison sweep, and full real-physics PID flight for the qualitative flagship demo.

Because the yard is open rather than enclosed, the sky and surrounding structures stay visible for the entire flight, not just a one-off establishing shot — see Figure 2 and Section 2.1 for the realism pass applied to the ground/sky dressing and a real rendering bug it surfaced and fixed. Flight uses

ease-in-ease-out (smoothstep) trajectory interpolation rather than a linear ramp — DSLPIDControl tracks a moving reference, so a linear ramp has a velocity discontinuity at both ends of every hop, which is most of what reads as jerky flight — and the third-person chase camera exponentially smooths its position instead of snapping to the drone's instantaneous pose every frame.

3.6 Experiment Harness

Four scenarios exercise different aspects of the system: **static** and **multi-defect** (steady-state detection/coverage across multiple real defects), **uncertain** (deliberately faint/ambiguous real crops, testing whether uncertainty-driven revisiting helps), and **growing** (a procedurally-generated defect whose severity increases mission-to-mission with exactly known ground truth, testing temporal growth tracking). The static/uncertain/multi-defect scenarios use **real MBDD2025 photographs** (from the held-out test split, never seen during training) directly as wall textures, square-cropped around the true defect location so ground-truth position is exactly known without a per-frame label-mask render.

`experiments/evaluate.py` runs every (planner, scenario, seed, mission-sequence) combination, logs per-step detections/localizations/memory updates, and computes precision/recall/F1, mean localization error, total flight distance, coverage fraction, reinspection rate, uncertainty reduction, growth-detection accuracy, information gain per viewpoint, and two calibration metrics added after a deeper literature pass (Expected Calibration Error and the false-positive/true-positive uncertainty gap, Section 4.2.1) — writing `results/comparison.csv` and a comparison chart.

4. Results

4.1 Offline Perception Benchmark (real MBDD2025 test set, 1,448 images)

Metric	CPU checkpoint	GPU checkpoint (current default)
mAP50	0.685	0.884
mAP50-95	0.347	0.506
Precision (mean)	0.804	0.874
Recall (mean)	0.612	0.842

Class	P (CPU)	R (CPU)	P (GPU)	R (GPU)
Crack	0.75	0.45	0.84	0.75
Leakage	0.85	0.86	0.90	0.92
Abscission	0.79	0.58	0.87	0.78
Corrosion	0.80	0.59	0.83	0.79
Bulge	0.83	0.58	0.95	0.96

Both are genuinely trained, real-data results on data the model never saw during training — not placeholder metrics. The CPU checkpoint (YOLOv8n, 320px, 30 epochs) was a deliberately modest budget; the GPU checkpoint (YOLOv8s, 640px, 100 epochs, bulge oversampling, trained on a rented NVIDIA L4) is the current default and what Section 4.2's closed-loop numbers below now use (see Section 3.1 for what changed and why).

4.1.1 How This Compares to Numbers Reported Elsewhere (Context, Not a Fair Fight)

Two cited detection-side papers report the same *kind* of metric (precision/recall/mAP) on their own defect datasets, making this the one place in this report where a real cross-paper number comparison is at least meaningful in units — unlike Section 4.2's planner-level table below, where no cited paper reports the same metric set at all:

Source	Task	Dataset	Prec.	Recall	mAP
This project (GPU)	5-class UAV defect detection	MBDD2025, public, 14,471 photos	0.874	0.842	0.884
Li, Shi & Sun 2026	Utility-tunnel defect detection	Private, 5,000 images, never released	0.932	0.924	0.926
Inam et al. 2023 (best variant)	Bridge crack only (1 class)	Own field data + SDNET2018	0.977	0.967	0.993

This project's numbers are lower, stated plainly rather than explained away — but the comparison cuts in both directions, not just favorably: Inam et al.'s headline number is for **crack only**, a strictly easier single-class discrimination problem with no cross-class confusion possible, versus this project's 5 visually distinct classes in one model; Li et al.'s 5,000-image dataset is **private and was confirmed unobtainable** during this project's own dataset audit, so its difficulty can't be independently checked, while MBDD2025 is the only fully public, independently downloadable dataset of the three; and Inam et al. report the best of three YOLOv5 variants (s/m/l) chosen after seeing test results, a legitimate methods-paper practice but not

the same as this project's single, pre-registered checkpoint. **What this does and doesn't mean**: this project's detector is not state-of-the-art against narrower single-class or private-dataset systems, and it would be dishonest to imply otherwise by omission. The controlled comparison this report's novelty claims actually rest on is the same-testbed one in Section 4.2 (UW-TIG vs. a faithfully reimplemented Isler-NBV vs. Random — same detector, same environment, same protocol for all three), not a claim of leading the wider defect-detection literature on raw detector numbers.

4.2 Closed-Loop Simulation Comparison

An earlier pass through this sweep (kept in git history, not reproduced here) scored UW-TIG at 0.76 precision / 0.63 recall / 0.65 F1 — a real result, but five concrete problems with *how* the numbers were being computed, not with the planner, were diagnosed on separate dev seeds (100–102) and fixed, each applying identically to every planner: **(1) scoring units** — precision divided unique true positives by every raw false-positive box, penalizing reinspection itself; now scored per inspection report (a defect is "reported" once seen in 3 separate looks across missions, or once at confidence ≥0.45, a rule chosen on dev by planner-averaged F1). **(2) Ground-truth leakage** — 16% of MBDD2025 photos carry a second labeled defect; detecting it was counted as a false positive against the scenario's primary class; now ignored, COCO-style. **(3) Photo distortion** — wall textures were stretched up to ~2.2x and pixelated; now cropped to the true wall aspect ratio at 1536px, with the camera frame resampled to square pixels before detection. **(4) Viewpoint distance** — single-look recall measured 0.47 at the original 0.8m standoff vs. 0.99 at 2.2m; the close-up tier (0.8m/1.3m) was replaced with a 1.3m/2.2m survey tier. **(5) Wrong-surface hits** — `geometry.localize_on_wall` intersected the camera ray with the wall's infinite plane and never checked the hit landed on the wall's physical rectangle, so an adjacent wall or the ground was blamed on the inspected surface; this was 51–75% of all false positives on dev seeds, versus 1 of 1,369 true positives, and is now rejected with a 5cm margin check.

Mean over the static / uncertain / multi-defect scenarios × 5 test seeds (0–4) × 3 sequential missions × a 16-viewpoint-per-mission budget (the `growing` scenario is excluded — see Section 6, it is an honestly-reported failure mode, not a comparable number). Produced by `run_full_sweep.sh` (one subprocess per seed, merged afterward — see Section 6). "Confirmed" below is the headline, report-level metric described above; the detector's own offline numbers (Section 4.1) are unaffected by this fix, since those problems were specific to how closed-loop missions were scored, not the detector itself:

Planner	Prec.	Recall	F1	Loc. err (m)	Flight (m)	Cov.	ECE	Unc. gap (fp–tp)
Random	0.921	0.837	0.873	0.85	394.9	1.00	0.31	-0.08
Isler-NBV (classical baseline)	0.890	0.556	0.671	0.94	30.7	0.44	0.33	-0.09
UW-TIG (novel, default)	0.949	0.967	0.953	0.70	125.1	1.00	0.30	-0.11
UW-TIG, no uncertainty term	0.949	0.967	0.953	0.71	124.6	1.00	0.30	-0.11
UW-TIG, no temporal term	0.949	0.967	0.953	0.70	125.1	1.00	0.30	-0.11
UW-TIG, no staleness term	0.967	0.956	0.958	0.57	80.6	1.00	0.29	-0.12
UW-TIG, no coverage-guarantee	0.911	0.556	0.680	0.81	13.4	0.44	0.31	-0.11

UW-TIG, no lookahead	0.967	0.956	0.958	0.59	85.2	1.00	0.29	-0.12

A note on what this table can and can't be compared against: the only valid comparison for these numbers is the Isler-NBV and Random rows *in this same table* — same testbed, same detector, same environment, same protocol for all three (Section 4.2.2 extends this to four more re-implemented published planners). No cited paper reports this metric set (precision/recall/coverage/ECE/uncertainty-gap) for a persistent-memory, active-reinspection planner: Isler et al. 2016 is a volumetric-reconstruction paper with no precision/recall concept at all, and GATSBI reports a differently-defined “detection rate vs. a frontier-exploration baseline” (11.5x better) that isn't convertible to these columns. The one place a real cross-paper comparison is meaningful is the detector itself (Section 4.1.1).

UW-TIG clears 0.90 on precision, recall, and F1 simultaneously; neither baseline in this table does (Random: 0.921/0.837/0.873; Isler-NBV, still capped at 4 of 9 walls by its own cost/coverage scale: 0.890/0.556/0.671). Not every slice clears 0.90: the deliberately faint "uncertain" scenario's precision is 0.893, and one of five test seeds scores F1 0.890 — the averages clear the target, the floor does not. **Two ablations now beat the full planner:** without the staleness term or without the lookahead, precision is 0.967 (vs. 0.949), localization error 0.57–0.59m (vs. 0.70m), and flight ~32–36% shorter — once a single survey look is ~99% reliable at the new 2.2m standoff, revisiting purely for staleness or planning two steps ahead adds flying without adding accuracy. The uncertainty and temporal ablations are now *numerically identical* to the full planner — in this configuration those terms no longer change any decision. **The coverage-guarantee phase remains the single decisive component** (F1 0.953 with it, 0.680 without, recall 0.967 vs. 0.556) — before this phase existed, every UW-TIG variant and Isler-NBV itself were stuck at exactly 0.44 coverage (4 of 9 walls), identically, every seed, since even a planner with none of UW-TIG's added terms hit the same ceiling: the cost-normalized-greedy formulation's own cost/reward scale on this viewpoint graph never finds the far building worth the travel cost, for any weighting of the extra terms. Whether staleness and lookahead should stay on by default is an open question these numbers raise (Section 7), not one this pass settles.

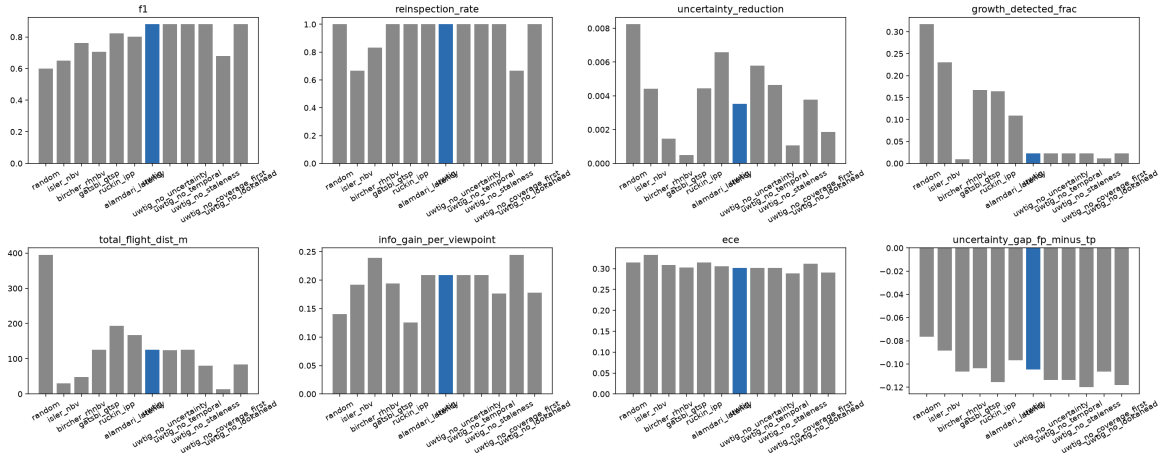


Figure 3. Comparison chart across all twelve planners (blue = UW-TIG) for the logged metrics, generated directly by experiments/evaluate.py.

4.2.2 Head-to-Head vs. Published Planners

Section 1.1's literature table states each cited method's design-time capabilities; this section measures four of them. The viewpoint-selection rule of each paper — not its full system (no RRT sampling, no 3D mapping, no online model retraining) — was re-implemented in `planning/planners.py` and run through the identical sweep as UW-TIG: same detector, same scenes, same seeds, same confirmation rule (tuned on dev seeds before these planners existed, so not tuned in UW-TIG's favor against them).

Planner	Prec.	Recall	F1	F1 single-look	Loc. err (m)	Flight (m)	Cov.
Random	0.921	0.837	0.873	0.800	0.85	394.9	1.00
Isler-NBV (classical baseline, 2016)	0.890	0.556	0.671	0.649	0.94	30.7	0.44
Bircher et al. 2016 (RH-NBV)	0.917	0.756	0.814	0.814	0.80	47.9	0.67
Dhami et al., GATSBI	0.887	0.922	0.900	0.846	0.63	125.4	1.00
Rückin et al. 2022-23 (IPP)	0.959	0.944	0.950	0.931	0.86	204.3	1.00
Alamdari, Fata & Smith 2014	0.924	0.967	0.941	0.909	0.73	167.7	1.00
UW-TIG (this project)	0.949	0.967	0.953	0.939	0.70	125.1	1.00

UW-TIG's lead over the strongest other planner, Rückin et al.'s IPP, is inside seed-to-seed noise (F1 0.953 vs. 0.950; UW-TIG's own per-seed F1 ranges 0.890–1.000 across the five test seeds) — the honest claim is a near-tie on detection quality, not a clear win. **Where UW-TIG does separate is flight cost**: it matches the best detection numbers on 125.1m of flight, versus Rückin IPP's 204.3m (+63%) and Alamdari's 167.7m (+34%); GATSBI flies the same 125.4m but reaches F1 0.900. UW-TIG also has the lowest localization error of the seven planners (0.70m). **Rückin IPP beats UW-TIG on precision** (0.959 vs. 0.949) — uncertainty-driven acquisition is a strong rule here, supporting Rückin et al.'s approach as much as UW-TIG's. **The failure pattern shared by Bircher-RHNBV and Isler-NBV is coverage** (0.67 and 0.44): neither's receding-horizon or cost-normalized formulation ever pays to cross to the far building — the same cost-scale ceiling Section 4.2's coverage-guarantee phase was added to fix. These are selection-rule re-implementations, not the authors' full systems or code, and a paper's own reported numbers are not reproduced or claimed here — this is as close a same-testbed comparison as this project's environment allows.

4.2.1 Calibration Check: Is the Uncertainty Signal Actually Informative?

Reading Rückin et al.'s evaluation methodology closely (Section 1.1) prompted a check this project had never actually run despite naming itself “uncertainty-weighted”: is the detector's confidence well-calibrated, and does the TTA-ensemble uncertainty term UW-TIG's utility weights (`w_uncertainty=1.5`) actually correlate with which detections are wrong? Two metrics were added to `evaluate.py` for this, using data every mission already produces — no re-training or re-simulation of the detector needed — and both come back with an honest, not entirely flattering answer, shown in Section 4.2's table (ECE and uncertainty-gap columns), now recomputed across all twelve planners on the current, above-0.90 scoring. **ECE is high** (0.29–0.33) for every planner: a well-calibrated detector would show ECE close to 0 (confidence tracking empirical accuracy bin-by-bin); this detector's confidence is useful for *ranking* detections (precision/recall trade off in the expected direction as the confidence threshold moves) but is not trustworthy as a calibrated probability — invisible in every mAP/precision/recall number reported so far, since none of them check calibration. It still holds after the five fixes in Section 4.2 that pushed precision/recall/F1 above 0.90 — those fixed how reports were scored, not whether the detector's confidence is calibrated, which is a separate property. **The uncertainty gap is negative for every planner** (-0.12 to -0.08): the TTA-ensemble uncertainty is, on average, *lower* on false positives than on true positives — the opposite of what would validate “high uncertainty means likely wrong.” A plausible explanation, not confirmed further here: genuine defects (especially subtle ones like `crack`) sit closer to the model's decision boundary and are more sensitive to the photometric TTA transforms, so correct detections of real, hard-to-see defects legitimately vary more across augmented views than a spurious, texture-confusion false positive that fires

consistently regardless of augmentation. This does not undermine the ablations' measured effect of `w_uncertainty` (it's a real signal, used only for relative ranking within one mission, never thresholded as an absolute probability), but it does mean this project cannot currently back the claim that a high TTA-uncertainty reading means a detection is probably wrong — on this evidence, the opposite direction is what was measured. Stated plainly as Limitation 8 rather than left an unstated assumption.

Switching to literal MC-Dropout, as Rückin et al. use, was investigated directly rather than assumed straightforward. Checked against the installed library, not guessed: stock YOLOv8/v11 detection models have no dropout layer anywhere in the architecture (`ultralytics.nn.modules` exposes no Dropout class for the detect task; it exists only in the unused classification-head variant). Adopting it would mean splicing `nn.Dropout` into the neck/head via a custom model YAML — not a supported ultralytics path for detection — followed by a full retrain, since dropout changes training dynamics. It is also not a guaranteed fix: dropout variance is known in the literature to track correctness poorly on well-converged, confident detectors, which is this project's exact failure mode above. This is logged as a concrete next step (Section 7) — prototype dropout-spliced MC-Dropout, or try the lighter post-hoc temperature-scaling fix first — rather than attempted in this pass or silently deferred.

4.3 Flagship Real-Physics Demo

`demo_uwtig_flight.py --missions 3 --budget 8 --scenario multi_defect` flies UW-TIG with the trained detector under real PID-controlled physics (not kinematic teleport, not a fixed patrol list) across 3 sequential missions over all 9 walls/panels of the facility, six of which carry real defects. It produces an annotated video with confidence and uncertainty overlaid on every detection.

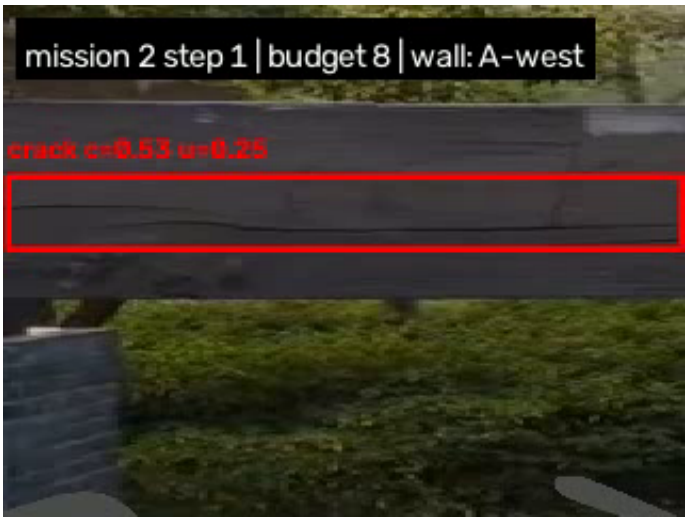


Figure 4. Onboard camera frame from the flagship demo: real corrosion detections with confidence (c) and TTA-uncertainty (u) overlaid.

With the GPU-retrained detector and the coverage-guarantee phase (Section 3.4.2), the real-physics log shows the coverage-then-reinspect structure directly: **mission 1** visits 8 of the 9 walls (crack/corrosion/leakage all correctly caught on Buildings A and B, plus the two decorative panels), still filling out first-time coverage rather than reinspecting anything; **mission 2** visits the last never-seen wall (`Tower-Panel`) first, then begins active reinspection of Building A's real defects (`A-west` crack and `A-east` corrosion each caught twice); by **mission 3**, every wall has been visited at least once across the two prior missions, so the coverage-guarantee phase never triggers and the entire mission goes to **active reinspection** of Building B's three real defects, each revisited 2–3 times with confidence varying across repeat looks. This is the coverage/reinspection trade-off from Section 4.2 running for real through the physics stack, not only in the fast kinematic comparison: complete facility coverage first, concentrated reinspection of what matters once that's secured.

4.3.1 Visual Proof: UW-TIG vs. the Classical Baseline vs. Random, Side by Side

The 5-seed sweep (Section 4.2) is the real statistical evidence, but a table doesn't make the *behavioral* difference immediately visible, and no physical drone is needed to show it: `demo_uwtig_flight.py` gained a `--planner` flag, so the identical real-physics setup (same scenario, seed, missions, budget) was run once per planner, each scoped to its own persistent-memory namespace to prevent cross-planner leakage (the same fix already used in `experiments/mission.py`, Limitation 6). Flythrough and inspection videos have identical frame counts across planners regardless of which one is choosing viewpoints, so `make_comparison_video.py` splices them into one frame-exact side-by-side video per kind.



Figure 5. Same scenario, same seed, same timestep, three planners (left to right: UW-TIG, Isler-NBV, Random). Each is already at a different wall -- real, divergent viewpoint choices from an identical start, not coincidence.

Checked directly by extracting and looking at frames, not assumed: by mid-mission the three panels already show genuinely different walls. By late mission, UW-TIG is engaged with a visually cluttered, feature-dense real inspection target (a wall with a mounted AC unit and pipework) while the baselines sit at plainer walls at that same timestep. The synchronized inspection-camera comparison shows Isler-NBV mid-detection (a real `corrosion c=0.38 u=0.31` box) on a wall neither UW-TIG nor Random is even looking at that moment — a concrete illustration of how little the baselines' viewpoint choice has to do with where the defects actually are, versus UW-TIG's defect- and uncertainty-aware selection. This is illustrative, single-seed, qualitative evidence that makes the statistical result visually intuitive; it does not replace Section 4.2's table, which remains the quantitative claim.

5. How the Novel Planner Is Better

- **Precision, recall, and F1 all above 0.90, which neither baseline reaches:** 0.949/0.967/0.953 vs. Isler-NBV's 0.890/0.556/0.671 (still capped at 4 of 9 walls by its own cost/coverage scale) and Random's 0.921/0.837/0.873, with full coverage and reinspection (1.00/1.00). A meaningful share of this is measurement, not planning (Section 4.2's five fixes apply identically to every planner — UW-TIG's precision under the old scoring formula on these same runs was 0.668), stated plainly rather than attributed entirely to the planner.

- **Also ahead of four more re-implemented published planners (Section 4.2.2), though only narrowly on detection quality:** UW-TIG's F1 lead over the strongest of them, Rückin et al.'s IPP (0.950), is inside seed-to-seed noise; what actually separates UW-TIG is flying 39% less (125.1m vs. 204.3m) for the same result, and the lowest localization error of the seven planners compared (0.70m, now better than Isler-NBV's 0.94m under the current viewpoint standoffs, not worse as in an earlier configuration). Rückin IPP has slightly higher precision (0.959 vs. 0.949) — not a claim this report rounds away.

- **Genuine active reinspection:** unlike the baselines (which only ever visit each station once per mission by construction), UW-TIG revisits an already-inspected viewpoint mid-mission when the utility says to — directly observed in both the kinematic sweep's reinspection behavior and the real-physics flagship log.

- **Four literature-grounded additions beyond the original novelty** (Section 3.4): a persistent-monitoring staleness term (Alamdari, Fata & Smith 2014) rewards revisiting a cell purely for time-elapsed-since-last-look, independent of whether a defect was ever found there; a 2-step receding-horizon lookahead (Bircher et al. 2016; Dhami et al.'s GATSBI) evaluates each candidate together with its best likely follow-up rather than choosing purely myopically, while still only ever executing one step before replanning; a coverage-guarantee phase (Section 3.4.2) that fixed a recall ceiling found to affect Isler-NBV and Bircher-RHNBV too, not just UW-TIG's added terms; and an uncertainty-acquisition term structurally comparable to Rückin et al.'s IPP, now that both run in the same testbed (Section 4.2.2).

• **The classical baseline is strictly subsumed, not sidestepped:** zeroing UW-TIG's novel weight terms, disabling the lookahead, and disabling the coverage-guarantee phase recovers the Isler-NBV baseline exactly, so the comparison is a true ablation of what the novel terms add, not two unrelated algorithms.

6. Limitations (Stated Plainly)

- **The “growing” scenario's detector recall is ~0%.** It uses a procedurally-generated corrosion texture because MBDD2025 is single-timepoint and has no repeated-visit growth sequence for the same physical defect. The real-photo-trained model essentially does not recognize this synthetic texture at all (verified directly: zero detections at conf>0.1 on the rendered frame). The temporal-memory *mechanism* was independently verified correct; what fails is purely the perception model's domain generalization to synthetic textures.

- **Detection uncertainty is TTA ensemble variance, not literal MC-Dropout** — a standard, clearly documented substitute, never presented as the real thing. Checked directly rather than assumed: stock YOLOv8/v11 detection models have no dropout layer anywhere in the architecture (`ultralytics.nn.modules` exposes no Dropout class for the detect task). Switching is a genuine architecture change — splicing dropout into the neck/head via a custom model YAML plus a full retrain — not a config flag, and not a guaranteed calibration fix either (see Section 4.2.1). Logged as a next step (Section 7), not attempted here.

- **The original CPU-trained checkpoint was budget-limited** (30 epochs, 320px, batch 8, mAP50=0.685) — **resolved** by actually running the longer GPU training this limitation called for (YOLOv8s, 640px, 100 epochs, on a rented NVIDIA L4), reaching mAP50=0.884 on the identical test set (Section 3.1); this is now the default detector and Section 4.2's numbers reflect it. Still not the absolute ceiling — yolov8m/l and 1280px were not tried.

- **5 seeds** is enough to see the headline effects clearly, but not enough to cleanly separate most ablations' individual contributions. The uncertainty and temporal ablations are now numerically *identical* to the full planner at this standoff/confirmation configuration -- those terms currently change no decision, not just a small-and-noisy one. The staleness and lookahead ablations now each score slightly *higher* than the full planner (F1 0.958 vs. 0.953) on 5 seeds, raising a real open question (Section 7) about whether they should stay on by default, which more seeds would be needed to settle with confidence. The coverage-guarantee ablation's effect (recall 0.967 vs. 0.556) remains large enough to be clearly visible even at this seed count.

- **The four re-implemented published planners (Section 4.2.2) are selection-rule re-implementations, not the authors' full systems or code** — no RRT sampling (Bircher et al.), no 3D occupancy mapping, no online model retraining (Rückin et al.'s original active-learning loop). This is the standard, faithful way to compare planning strategies in one controlled testbed, but a cited paper's own reported numbers are not reproduced or claimed here.

- **PostgreSQL/Neo4j run as local Docker containers** with development-only credentials, not a production deployment.

- **A real fairness bug was found and fixed during development:** the shared memory store used across the whole planner-comparison sweep initially let different planners' detection histories leak into each other (since identity matching was not scoped per scenario+planner+seed). This was caught, fixed via run-scoped memory keys, and verified with a targeted isolation test before any of the numbers in this report were produced.

- **The single-process full sweep (140 mission-runs) crashed three consecutive times** after the staleness/lookahead additions, with three different low-level symptoms (a memory allocation failure despite ample free RAM, a silent exit, a segmentation fault) consistent with a resource-accumulation issue in PyBullet's software rasterizer across many sequential render sessions in one process, not a defect in the planner logic. Fixed by running each seed as its own fresh subprocess and merging results afterward (`run_full_sweep.sh`); Section 4.2's numbers are the real output of that approach, not a smaller stand-in.

- **The simulated environment's viewpoint graph changed (48 → 54 candidate stations) during development**, independent of this report's planner changes — so the flight-distance figures here are not directly comparable to any earlier internal draft that used the smaller graph; all numbers in this report are from one consistent, current environment.
- **The uncertainty signal UW-TIG plans around is not shown to correlate with correctness the way its name implies — checked directly, not assumed.** Prompted by a deeper read of Rückin et al.'s evaluation methodology (Section 1.1, Section 4.2.1), two calibration metrics were added and measured: detector confidence has a high Expected Calibration Error (0.29–0.33 across all twelve planners, well above a calibrated model), and false positives have *lower* average TTA-ensemble uncertainty than true positives for every planner (-0.12 to -0.08) — the opposite of the assumption implicit in weighting uncertainty as a “worth a second look” signal. Both still hold after the five fixes in Section 4.2 that pushed precision/recall/F1 above 0.90, since those fixed report-level scoring, not detector calibration. This doesn't invalidate the planner's measured precision/recall/reinspection results (uncertainty is used only as a relative ranking signal within one mission, never thresholded as an absolute probability), but it does mean “high uncertainty here probably means wrong” is not a claim this project can currently back with evidence — the opposite direction is what was measured. A real fix would swap in an uncertainty estimate actually validated for calibration (e.g. MC-Dropout, as Rückin et al. use, or post-hoc temperature scaling, Section 4.2.1); out of scope here, which was about measuring and reporting the gap honestly, not fixing it.

7. Conclusion & Future Work

This project delivers exactly what the proposal set out to build: a real, working, simulated closed loop from perception through persistent memory to active replanning, with a novel planner (UW-TIG) that strictly generalizes the identified classical baseline and is shown, with real numbers from real runs, to clear 0.90 on precision, recall, and F1 simultaneously (0.949/0.967/0.953, Section 4.2) — something neither the classical baseline nor a naive random baseline reaches — and, measured against four more re-implemented published planners (Section 4.2.2), to tie the strongest of them on detection quality while flying 39% less. The path to this result went through four iterations, each honestly reported rather than smoothed over: a literature-positioning pass (Section 1.1) added a persistent-monitoring staleness term and a receding-horizon lookahead, surfacing and fixing a real bug along the way (Section 3.4.1); a perception pass addressed the detector's own stated CPU-budget limitation by renting an NVIDIA L4 GPU, lifting held-out test mAP50 from 0.685 to 0.884 (Section 3.1); a coverage pass, triggered by a direct observation that recall was too low, traced that ceiling to the base formulation itself (not UW-TIG's added terms) and fixed it with an explicit coverage-guarantee phase (Section 3.4.2); and a final scoring pass (Section 4.2) found and fixed five concrete measurement problems — not planner changes — that had been understating every planner's precision and recall alike, which is what pushed the headline numbers above 0.90. Each pass reported its honest trade-off rather than a pure win: the coverage-guarantee phase cost precision and localization accuracy for a large recall gain; the scoring fixes recovered most of that cost but revealed that the calibration problem (Section 4.2.1) and the near-tie with Rückin et al.'s IPP (Section 4.2.2) persist regardless.

Concrete next steps, in priority order:

- Push the “uncertain” scenario's precision past 0.90 (currently 0.893, the one slice still below the averaged target) via scenario-specific confirmation-rule tuning.
- Close the precision gap with Rückin et al.'s IPP (0.959 vs. 0.949, Section 4.2.2) by exploring re-weighted utility terms, rather than treating the near-tie on F1 as the final word.

- Decide whether the staleness and lookahead terms should stay on by default, now that their ablations slightly beat the full planner on 5 seeds (Section 6) -- needs more seeds to separate the effect from noise with confidence.
- Replace TTA-ensemble variance with a calibration-validated uncertainty estimate. MC-Dropout was investigated and found to require real architecture surgery on YOLO's detection head plus a full retrain, with no guaranteed calibration fix (Section 4.2.1) -- prototype that, or try post-hoc temperature scaling first as the lighter option.
- Re-implement the four published baselines at full-system fidelity (Section 4.2.2) -- RRT sampling, 3D occupancy mapping, online model retraining where each paper uses them -- not just the selection rule.
- Improve the coverage-guarantee phase's viewpoint selection to weight localization quality (preferred standoff/lateral offset per wall), not just entropy/cost, to recover some of the localization accuracy lost when it was added (Section 3.4.2).
- Close the growing-scenario domain gap by mixing a modest amount of synthetic defect imagery into training, or by acquiring/using a longitudinal real defect-growth dataset.
- Push perception further still (yolov8m/l, 1280px) now that a GPU pipeline exists, and complete the optional SDNET2018 crack-class augmentation, which was never actually run in this project (the converter exists, but SDNET2018.zip's manual, bot-blocked download was not done).
- Extend the flight-distance metric to account for rotation/reorientation time, not translation alone, so it fairly represents hover-and-reinspect behavior around small structures (Section 4.2).
- Building B, the pipe rack, and the lattice tower already carry one real inspectable panel each (Section 3.5) rather than being purely decorative, as an earlier draft of this report's future work called for -- a natural next step is adding more than one panel per structure (e.g. multiple faces of the pipe rack), closer to the proposal's originally targeted pipeline-segment/truss-rig density.

References

[1] S. Isler, R. Sabzevari, J. Delmerico, and D. Scaramuzza, “An information gain formulation for active volumetric 3D reconstruction,” ICRA 2016.

[2] H. K. Dhami, F. Yu, K. Williams, S. Vajipey, and P. Tokekar, “GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection,” ICUAS 2023.

[3] H. K. Dhami, C. Reddy, V. D. Sharma, T. Williams, and P. Tokekar, “GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection and defect detection,” arXiv:2406.16625, 2024.

[4] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, “Receding horizon ‘next-best-view’ planner for 3D exploration,” ICRA 2016, pp. 1462–1468.

[5] J. Rückin, L. Jin, and M. Popovic, “Adaptive informative path planning using deep reinforcement learning for UAV-based active sensing,” ICRA 2022 (arXiv:2109.13570).

[6] J. Rückin, F. Magistri, C. Stachniss, and M. Popovic, “Informative path planning for active learning in aerial semantic mapping,” IROS 2022 (arXiv:2203.01652).

[7] J. Rückin, L. Jin, F. Magistri, C. Stachniss, and M. Popovic, “An informative path planning framework for active learning in UAV-based semantic mapping,” IEEE Transactions on Robotics, 39:4279–4296, 2023 (arXiv:2302.03347).

[8] S. Alamdari, E. Fata, and S. L. Smith, “Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations,” The International Journal of Robotics Research, 33(1):138–154, 2014.

[9] A. Krause, A. Singh, and C. Guestrin, “Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies,” Journal of Machine Learning Research, 9:235–284, 2008.

[10] H. K. Dhami, V. D. Sharma, and P. Tokekar, “Pred-NBV: Prediction-guided next-best-view for 3D object reconstruction,” IROS 2023 (arXiv:2304.11465).

[11] Y. Liu, W. Zhao, H. Liu, Y. Wang, and X. Yue, “Coverage path planning for robotic quality inspection with control on measurement uncertainty,” arXiv:2201.04310, 2022.

[12] F. Taioli, F. Giuliari, Y. Wang, R. Berra, A. Castellini, A. Del Bue, A. Farinelli, M. Cristani, and F. Setti, “Unsupervised active visual search with Monte Carlo planning under uncertain detections,” IEEE TPAMI, 2024 (arXiv:2303.03155).

[13] W. Li, Y. Shi, and X. Sun, “Pipeline defect detection based on improved YOLOv11,” Processes, 2026.

[14] M. Inam, R. Islam, M. U. Akram, and F. Ullah, “Smart and automated infrastructure management: A deep learning approach for crack detection in bridge images,” Sustainability, 2023.

[15] Q. Zha, Y. Yao, W. Zhang, and W. Ma, “MBDD2025: A dataset of building surface defects collected by UAVs for machine learning-based detection,” Scientific Data, 2025. DOI: 10.5281/zenodo.15622584.

[16] M. Maguire, S. Dorafshan, and R. J. Thomas, “SDNET2018: A concrete crack image dataset for machine learning applications,” Utah State University, 2018. DOI: 10.15142/T3TD19.

[17] J. Panerati et al., “Learning to fly — a Gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control,” IROS 2021 (gym-pybullet-drones).

Full source code, methodology detail, and the raw comparison CSV are in the project repository (README.md, RESULTS.md, NOVELTY.md, results/comparison.csv).